

Contrato da API: Tarefa Node.js (Gerenciamento de Projetos)

Este documento define **exatamente** os endpoints, formatos de request/response e códigos HTTP que sua API precisa implementar. A avaliação automática é feita em cima deste contrato: divergências de path, verbo HTTP ou formato de resposta fazem o teste correspondente falhar, mesmo que a lógica de negócio esteja correta.

Em caso de dúvida sobre uma regra de negócio, o documento original da tarefa (`Tarefa de Node versão final.pdf`) é a fonte da verdade sobre o *comportamento*; este contrato é a fonte da verdade sobre o *formato* (rotas, payloads, status codes).

Os objetos de resposta não são um espelho 1:1 do seu modelo/tabela. Cada seção abaixo mostra exatamente os campos que devem (e não devem) aparecer em cada resposta. O caso mais óbvio é a senha do usuário: o hash dela fica só no banco, e não é serializado em nenhuma resposta da API, nem em `POST /auth/register`, nem em `GET/PUT /users/:id`, nem em nenhum outro lugar. Se o seu ORM/serializer devolve o objeto inteiro por padrão, filtre explicitamente os campos internos antes de responder. O campo `publicId`, por outro lado, é **opcional**: se você optou por implementá-lo no seu schema, tudo bem que ele apareça nas respostas. A avaliação não penaliza a presença dele, só a de campos sensíveis como senha/hash.

Sumário

- [Ambiente de execução](#)
- [Autenticação e controle de acesso \(RBAC\)](#)
- [Usuários \(`users` \)](#)
- [Projetos \(`projects` \)](#)
- [Tarefas \(`tasks` \)](#)
- [Atribuição de tarefas](#)
- [Relatórios \(`reports` \)](#)
- [Filtros e ordenação](#)

- [CORS](#)
- [Formato de erros](#)
- [Resumo de status HTTP esperados](#)

Ambiente de execução

Requisitos técnicos

- Usar **Prisma** como ORM, com o schema em `prisma/schema.prisma` e as migrations commitadas em `prisma/migrations/` (geradas com `npx prisma migrate dev` durante o seu desenvolvimento). A avaliação automática roda `npx prisma migrate deploy` antes de subir sua API. Você **não** precisa (nem deve) rodar migrations dentro do seu `npm start`, ele deve apenas iniciar o servidor HTTP assumindo que o schema já foi aplicado.
- Expor `GET /health` retornando `200 { "status": "ok" }` assim que o servidor e o banco estiverem prontos para receber requisições. Esse endpoint é usado só para saber quando sua API está de pé, não é um requisito funcional da tarefa em si.
- Hash de senha com **bcrypt** (ou compatível), conforme pedido no enunciado.
- JWT para autenticação; os dados do usuário autenticado (id e role, no mínimo) devem estar disponíveis no request após o middleware de autenticação validar o token.

Variáveis de ambiente

Copie `.env.example` (na raiz do repositório) para `.env` e ajuste se quiser. Os valores de exemplo já são os mesmos usados pela avaliação automática, então rodando com eles seu ambiente local fica idêntico ao de CI.

Variável	Obrigatória p/ contrato	Descrição
<code>PORT</code>	sim	Porta em que a API escuta. Padrão <code>3000</code> se não definida.
<code>DATABASE_URL</code>	sim	String de conexão do Postgres, formato <code>postgresql://<usuario>:<senha>@<host>:<porta>/<database>?schema=public</code> , no formato que <code>env("DATABASE_URL")</code> do Prisma espera.
<code>JWT_SECRET</code>	sim	Segredo usado para assinar/validar o JWT.

Variável	Obrigatória p/ contrato	Descrição
NODE_ENV	não	development localmente; a avaliação roda com production .
HOST	não	Host de bind do servidor. Sugestão: 0.0.0.0 .
HASH_SALT_ROUNDS	não	Rounds de salt para o bcrypt.
CORS_ORIGIN	não	Sugestão de origem liberada no CORS (ex: *).
POSTGRES_HOST	não*	Host do Postgres, usado pelo docker-compose.yml .
POSTGRES_USER	não*	Usuário do Postgres, usado pelo docker-compose.yml .
POSTGRES_PASSWORD	não*	Senha do Postgres, usada pelo docker-compose.yml .
POSTGRES_DB	não*	Nome do banco, usado pelo docker-compose.yml .
POSTGRES_PORT	não*	Porta exposta do Postgres, usada pelo docker-compose.yml .
SCHEMA	não*	Schema do Postgres usado no DATABASE_URL (? schema=public).

* Essas variáveis não são lidas pela sua aplicação diretamente, elas alimentam o docker-compose.yml para montar o Postgres local. O que sua aplicação de fato usa para conectar é o DATABASE_URL já montado.

Use o docker-compose.yml deste repositório para subir um Postgres local com as mesmas credenciais usadas na avaliação automática, e rode npx prisma migrate dev você mesmo localmente para gerar/aplicar suas migrations durante o desenvolvimento.

Autenticação e controle de acesso (RBAC)

Todas as rotas são protegidas por JWT, **exceto** POST /auth/register e POST /auth/login . Existem dois papéis (role):

- admin** : acesso total a todas as rotas.
- user** (papel padrão): leitura das rotas marcadas como Auth mais ações sobre os próprios dados (ver seções de Usuários e Tarefas abaixo).

Método	Rota	Auth	Descrição
POST	/auth/register	não	Cadastrar usuário
POST	/auth/login	não	Login e geração de JWT

Objeto Usuário (resposta):

```
{
  "id": 1,
  "name": "string",
  "email": "string",
  "role": "admin | user",
  "createdAt": "ISO8601",
  "updatedAt": "ISO8601"
}
```

Note que não há campo de senha nem de hash. Nenhum endpoint retorna isso, em nenhuma hipótese (nem `password` em texto puro, nem o hash armazenado no banco).

POST /auth/register

Body:

```
{
  "name": "string",
  "email": "string",
  "password": "string (mínimo 6 caracteres)",
  "role": "admin | user (opcional, padrão \"user\")"
}
```

- `201` : Objeto Usuário (ver acima)
- `400` : campo obrigatório ausente, e-mail em formato inválido, senha com menos de 6 caracteres, ou `role` fora do enum (`admin` / `user`)
- `409` : e-mail já cadastrado

Nota de design: por que `role` é aceito no registro. Numa aplicação real, promover alguém a admin não seria algo que o próprio registro público permite. Mas como a avaliação automática é 100% black-box via HTTP (sem acesso direto ao banco do candidato), o único jeito de ela conseguir testar as rotas `Admin` é criando sua própria conta admin através da

API. Por isso o campo `role` é aceito (e opcional, com padrão `"user"`) diretamente em `POST /auth/register`. Implemente exatamente como descrito: se `role` vier no corpo e for um valor válido, use-o; se não vier, o usuário criado deve ser `"user"`.

POST /auth/login

Body: `{ "email": "string", "password": "string" }`

- `200`: `{ "token": "string" }` (JWT)
- `401`: credenciais inválidas

Regras gerais de autorização

- Rotas protegidas exigem o header `Authorization: Bearer <token>`.
 - Header ausente ou token inválido/expirado retorna `401`.
- Quando a rota exige um papel específico (`Admin`, ou "admin ou dono do recurso") e o usuário autenticado não atende a condição, retorna `403`.
- A ordem de verificação é sempre: token ausente/inválido retorna `401` antes de checar papel; token válido mas papel insuficiente retorna `403`.

Usuários (users)

Método	Rota	Acesso	Descrição
GET	<code>/users</code>	Auth	Listar usuários
GET	<code>/users/:id</code>	Auth	Buscar usuário por id
PUT	<code>/users/:id</code>	Admin ou próprio usuário	Atualizar usuário
DELETE	<code>/users/:id</code>	Admin	Deletar usuário
GET	<code>/users/:id/tasks</code>	Auth	Listar tarefas atribuídas ao usuário

- `GET /users`: `200` com array de Objeto Usuário.
- `GET /users/:id`: `200` com Objeto Usuário, ou `404` se não existir.
- `PUT /users/:id`
 - **Body:** `{ "name"?: "string", "password"?: "string" }`

- O campo `email` não pode ser alterado por esta rota (regra explícita da tarefa). Se vier no corpo, deve ser **ignorado silenciosamente**: a resposta reflete o e-mail original, sem erro.
- Acesso: o próprio usuário (dono do `:id`) ou um `admin`. Qualquer outro usuário autenticado recebe `403`.
- `200`: Objeto Usuário atualizado
- `400`: dados inválidos (ex: senha nova com menos de 6 caracteres)
- `401` / `403` / `404`
- `DELETE /users/:id`
 - Acesso: `admin` apenas.
 - Ao deletar, os vínculos do usuário em `TaskUser` (tarefas atribuídas a ele) devem ser removidos automaticamente. A deleção do usuário não deve falhar nem exigir que ele esteja "livre" de tarefas primeiro.
 - `204` / `401` / `403` / `404`
- `GET /users/:id/tasks`: `200` com array de Objeto Tarefa (as tarefas às quais esse usuário está atribuído via `TaskUser`), ou `404` se o usuário não existir.

Projetos (`projects`)

Método	Rota	Acesso	Descrição
GET	<code>/projects</code>	Auth	Listar projetos
GET	<code>/projects/:id</code>	Auth	Buscar projeto por id
POST	<code>/projects</code>	Admin	Criar projeto
PUT	<code>/projects/:id</code>	Admin	Atualizar projeto
DELETE	<code>/projects/:id</code>	Admin	Deletar (se não houver tarefas associadas)
GET	<code>/projects/:id/tasks</code>	Auth	Listar tarefas do projeto

Objeto Projeto:

```
{
  "id": 1,
  "name": "string",
```

```
"description": "string | null",
"status": "active | completed | cancelled",
"createdAt": "ISO8601",
"updatedAt": "ISO8601"
}
```

- **POST /projects** : body { "name" (obrigatório), "description"?, "status"? }, retorna **201** com o objeto criado.
 - **name** ausente retorna **400**
 - **status** fora do enum (**active** / **completed** / **cancelled**), quando informado, retorna **400**
 - Acesso: **admin** apenas. **403** para **user** autenticado, **401** sem token
- **GET /projects** : **200** com array de Objeto Projeto.
- **GET /projects/:id** : **200** com o objeto, ou **404** se não existir.
- **PUT /projects/:id** : **200** com o objeto atualizado, ou **404** . **admin** apenas.
- **DELETE /projects/:id** : **admin** apenas.
 - **204** se não houver tarefas associadas ao projeto.
 - **409** se existir pelo menos uma tarefa associada (regra explícita da tarefa: projeto com tarefas não pode ser removido).
 - **404** se o projeto não existir.
- **GET /projects/:id/tasks** : **200** com array de Objeto Tarefa do projeto, ou **404** se o projeto não existir.

Tarefas (tasks)

Objeto Tarefa:

```
{
  "id": 1,
  "title": "string",
  "description": "string | null",
  "priority": "low | medium | high",
  "completed": false,
  "deadline": "ISO8601 | null",
  "projectId": 1,
```

```

"createdAt": "ISO8601",
"updatedAt": "ISO8601",
"assignedUsers": [
  { "id": 1, "name": "string", "email": "string", "role": "admin | user" }
]
}

```

`assignedUsers` reflete a tabela associativa `TaskUser` e deve vir preenchido pelo menos em `GET /tasks/:id` e nas respostas dos endpoints de atribuição/remoção descritos a seguir.

Método	Rota	Acesso	Descrição
GET	<code>/tasks</code>	Auth	Listar tarefas (com filtros/ordenação)
GET	<code>/tasks/:id</code>	Auth	Buscar tarefa por id
POST	<code>/tasks</code>	Admin	Criar tarefa
PUT	<code>/tasks/:id</code>	Admin	Atualizar tarefa
DELETE	<code>/tasks/:id</code>	Admin	Deletar tarefa
PATCH	<code>/tasks/:id/complete</code>	Admin ou usuário atribuído	Marcar tarefa como concluída

- `POST /tasks`: body { "title" (obrigatório), "description"?, "priority"?, "deadline"?, "projectId" (obrigatório) }, retorna 201.
 - `title` ausente retorna 400
 - `priority` fora do enum (`low` / `medium` / `high`), quando informado, retorna 400
 - `projectId` ausente retorna 400 ; `projectId` de projeto inexistente retorna 404
 - Acesso: `admin` apenas
- `GET /tasks`: 200 com array. Aceita query params combináveis (ver [Filtros e ordenação](#)).
- `GET /tasks/:id`: 200 (com `assignedUsers`) ou 404.
- `PUT /tasks/:id`: 200 com o objeto atualizado, ou 404. `admin` apenas.
- `DELETE /tasks/:id`: `admin` apenas.
 - 204. Remove automaticamente as associações da tarefa em `TaskUser` (não deve falhar por a tarefa ter usuários atribuídos).
 - 404 se a tarefa não existir.

- `PATCH /tasks/:id/complete` : marca `completed: true` , retorna `200` com o objeto atualizado.
 - Acesso: `admin` ou um `user` que esteja atribuído à tarefa (presente em `TaskUser` para essa tarefa). Qualquer outro `user` autenticado (não atribuído) recebe `403` .
 - `401` sem token; `404` se a tarefa não existir.

Atribuição de tarefas

Método	Rota	Acesso	Descrição
POST	<code>/tasks/:id/assign</code>	Admin	Atribuir usuários à tarefa
DELETE	<code>/tasks/:id/assign/:userId</code>	Admin	Remover usuário da tarefa

- `POST /tasks/:id/assign` : `body { "userIds": [1, 2] }` , retorna `200 / 201` com a tarefa atualizada (incluindo `assignedUsers`).
 - `400` se `userIds` ausente/vazio
 - `404` se a tarefa não existir ou se algum `userId` não corresponder a um usuário existente
 - Atribuir um usuário que já está atribuído não deve duplicar a associação nem falhar.
- `DELETE /tasks/:id/assign/:userId` : remove a associação, retorna `204` .
 - `404` se a tarefa, o usuário, ou a associação entre eles não existir.

Relatórios (reports)

`GET /reports/projects`

Acesso: `admin` apenas (`401` sem token, `403` para `user`).

- `200` com array, um item por projeto:

```
[
  {
    "projectId": 1,
    "name": "string",
    "totalTasks": 10,
    "completedTasks": 4,
    "completionPercentage": 40
  }
]
```

```
}  
]
```

`completionPercentage` é `completedTasks / totalTasks * 100` (use `0` quando `totalTasks` for `0`, para evitar divisão por zero).

Filtros e ordenação

Aplicáveis a `GET /tasks`, combináveis entre si:

- `?completed=true|false` : filtra pelo campo `completed`
- `?priority=low|medium|high` : filtra pelo campo `priority`
- `?sort=<campo>&order=asc|desc` : ordena pelo campo informado (ex: `sort=deadline&order=desc`)

CORS

A API deve responder com os headers de CORS liberando requisições de qualquer origem (`Access-Control-Allow-Origin` , no mínimo, presente nas respostas).

Formato de erros

Toda resposta de erro (`400` , `401` , `403` , `404` , `409`) deve ser um JSON com pelo menos:

```
{ "message": "descrição legível do erro" }
```

Resumo de status HTTP esperados

Situação	Status
Recurso criado com sucesso	201
Leitura/atualização com sucesso	200
Deleção com sucesso	204
Validação falhou (campo obrigatório, formato, enum inválido, senha curta)	400

Situação	Status
Ausência/invalidadez de token em rota protegida	401
Token válido, mas papel/dono insuficiente para a ação (RBAC)	403
Recurso não encontrado	404
Conflito (deletar projeto com tarefas; e-mail duplicado)	409